Optimisation Discrète : Projet Course d'avions

Thomas Chrétienne et Tom Cuel

1 Modèles

Données et variables

On note $V = \{1, \dots, n\}$ les aérodromes, d_{ij} la distance entre i et j, DEP et ARR les aérodromes de départ et d'arrivée, A_{min} le nombre minimal d'aérodromes à visiter, K le nombre de régions, $R_i \in \{0, \dots, K\}$ la région de l'aérodrome i et R la distance maximale d'un trajet sans se poser.

Variables : $x_{ij} \in \{0, 1\}$ vaut 1 si l'avion vole directement de i à j.

Contraintes communes aux deux modèles

$$
\begin{aligned}
&\min \ \textstyle\sum_{i=1}^{n} \sum_{j=1}^{n} d_{ij}\, x_{ij} \\
&\text{s.c. } x_{ii} = 0 && i \in V && (1)\\
&\quad d_{ij}\, x_{ij} \le R && i, j \in V && (2)\\
&\quad \textstyle\sum_{j=1}^{n} x_{DEP,j} = 1, \qquad \sum_{i=1}^{n} x_{i,DEP} = 0 && && (3)\\
&\quad \textstyle\sum_{i=1}^{n} x_{i,ARR} = 1, \qquad \sum_{j=1}^{n} x_{ARR,j} = 0 && && (4)\\
&\quad \textstyle\sum_{j=1}^{n} x_{ij} = \sum_{j=1}^{n} x_{ji} && i \neq DEP,\ ARR && (5)\\
&\quad \textstyle\sum_{j=1}^{n} x_{ij} \le 1 && i \neq DEP,\ ARR && (6)\\
&\quad \textstyle\sum_{i=1}^{n} \sum_{j=1}^{n} x_{ij} \ge A_{min} - 1 && && (7)\\
&\quad \textstyle\sum_{j\,:\,R_j = k} \ \sum_{i=1}^{n} x_{ij} \ge 1 && k = 1, \dots, K \qquad k \neq R_{\mathrm{DEP}} && (8)\\
&\quad x_{ij} \in \{0, 1\} && i, j \in V &&
\end{aligned}
$$

(1) : on ne vole pas d'un aérodrome vers lui-même.
(2) : on ne fait pas de trajet plus long que R sans se poser.
(3) : on quitte une fois l'aérodrome de départ et on n'y revient jamais.
(4) : on arrive une fois à l'aérodrome d'arrivée et on n'en repart jamais.
(5) : un aérodrome intermédiaire est quitté autant de fois qu'on y arrive.
(6) : un aérodrome intermédiaire est quitté au plus une fois, donc visité au plus une fois.
(7) : un parcours qui passe par A_{min} aérodromes utilise $A_{min} - 1$ trajets.
(8) : au moins un trajet arrive dans chaque région k (la région du départ est déjà visitée, la région 0 n'est pas imposée).

Ces contraintes n'empêchent pas les sous-tours, c'est-à-dire des cycles séparés du chemin de DEP à ARR. Les deux modèles diffèrent par la façon de les interdire.

Modèle 1 : nombre polynomial de contraintes

On ajoute des variables u_i (position de l'aérodrome i dans le parcours) et les contraintes :

$$
\begin{aligned}
&u_j \ge u_i + 1 - n\,(1 - x_{ij}) && i, j \in V,\ i \neq j \\
&1 \le u_i \le n && i \in V
\end{aligned}
$$

Si $x_{ij} = 1$, alors $u_j \ge u_i + 1$: les positions augmentent le long du parcours, donc aucun cycle n'est possible. Si $x_{ij} = 0$, la contrainte est toujours vérifiée.

Modèle 2 : nombre exponentiel de contraintes

On ajoute les contraintes d'élimination des sous-tours (il y en a de l'ordre de 2^n, on ne peut donc pas toutes les écrire) :

$$\sum_{i \in S} \sum_{j \in S} x_{ij} \le |S| - 1 \qquad S \subseteq V,\ |S| \ge 2.$$

Problème de séparation : Étant donnée une solution x^* du problème sans ces contraintes, on cherche un ensemble S tel que $\sum_{i,j \in S} x^*_{ij} > |S| - 1$. Si x^* est entière, il suffit de suivre les trajets choisis : on obtient le chemin de DEP à ARR et éventuellement des cycles. Chaque cycle donne un ensemble S violé,

dont on ajoute la contrainte avant de résoudre à nouveau. On s'arrête quand la solution ne contient plus de cycle.

2 Mise en œuvre

Nous avons implémenté les deux modèles en Julia avec JuMP et Gurobi, avec une limite de 10 minutes par résolution.

— **Modèle 1** : on résout d'abord la relaxation continue (`relax_integrality`) pour obtenir la borne, puis le problème en nombres entiers avec le Branch-and-Cut de Gurobi.

— **Modèle 2** : on résout le problème sans contraintes de sous-tours, on ajoute une contrainte pour chaque cycle trouvé et on recommence. Le temps et le nombre de noeuds sont additionnés sur toutes les résolutions. La borne donnée est celle de la relaxation continue du modèle final, c'est-à-dire avec les contraintes de sous-tours ajoutées pendant la résolution.

3 Résultats

Pour chaque instance et chaque modèle : borne par relaxation continue, temps et nombre de noeuds du Branch-and-Cut. La valeur optimale est la même pour les deux modèles ; > 600 indique la limite de 10 minutes atteinte et * la meilleure solution trouvée.

			Modèle 1 (MTZ)			Modèle 2 (sous-tours)		
Instance	n	Valeur	Borne	Temps (s)	noeuds	Borne	Temps (s)	noeuds
0 (exemple)	6	12	8.0	0.01	1	8.0	0.00	4
20_1	20	73	65.2	0.03	1	66.5	0.02	3
20_2	20	91	63.0	0.06	1	63.0	0.12	11
30_1	30	115	84.1	0.13	737	89.0	1.42	3 900
40_1	40	124	114.2	0.07	1	118.0	0.05	8
50_1	50	148	137.0	0.09	1	139.0	0.07	6
70_1	70	288	231.1	1.05	3 358	238.0	4.19	1 573
80_1	80	385	221.3	58.03	336 621	263.0	436.23	2 336 678
90_1	90	608*	409.8	> 600	2 205 064	–	> 600	2 364 356
100_1	100	665	429.5	112.82	257 474	518.0	312.61	885 954
100_2	100	580	410.9	26.43	77 599	–	> 600	2 409 180
120_1	120	426*	248.5	> 600	795 927	–	> 600	1 984 119

— Jusqu'à 50 aérodromes, tout est résolu en moins de 2 secondes, souvent dès la racine.

— La borne du modèle 2 est toujours au moins aussi bonne que celle de MTZ (par exemple 518 contre 429.5 sur 100_1, pour un optimum de 665) : le big-M de MTZ affaiblit la relaxation continue.

— Pourtant, sur les grandes instances, le modèle 2 est plus lent, car nous relançons une résolution complète à chaque ajout de sous-tours. À partir de 90 aérodromes, plusieurs instances ne sont pas résolues en 10 minutes.

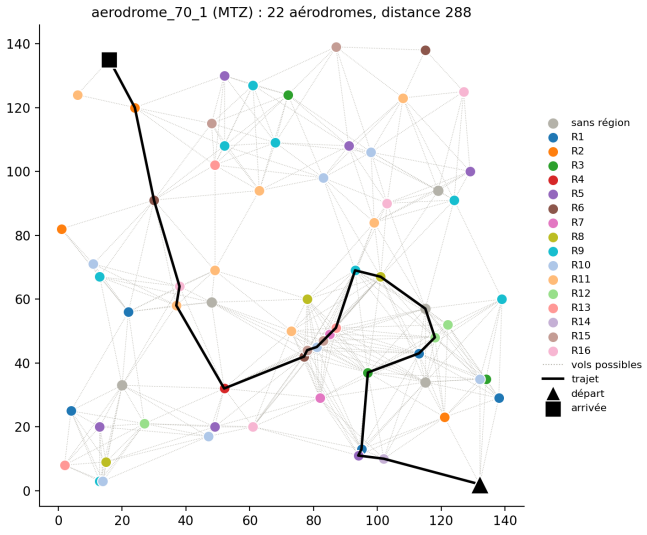


FIGURE 1 – Parcours optimal sur l'instance 70_1 (22 aérodromes, distance 288). Une couleur par région, aérodromes sans région en gris, vols possibles ($d_{ij} \leq R$) en pointillés

4 Instances avec des régions regroupées

Dans les instances fournies, les aérodromes d'une même région sont répartis sur toute la carte (figure 2, à gauche). Nous avons construit deux instances où chaque région forme un groupe d'aérodromes proches, avec quelques aérodromes sans région (figure 2, à droite).

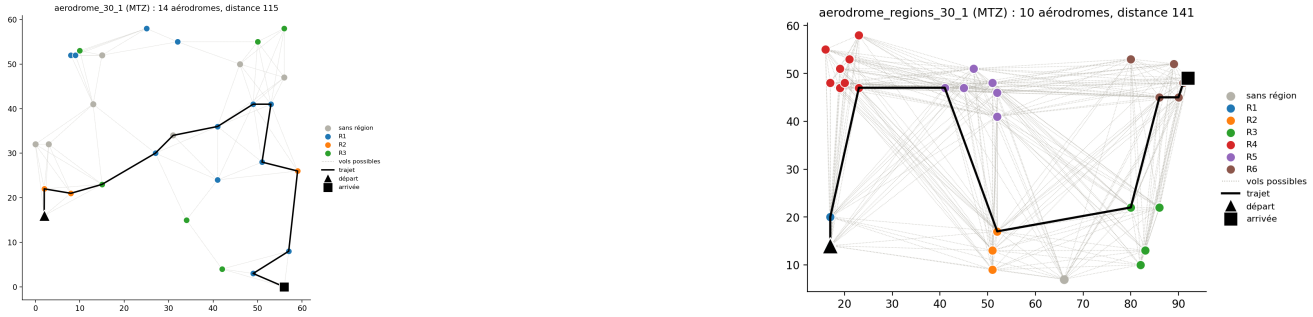


FIGURE 2 – À gauche, instance fournie 30_1 : les régions sont mélangées (3 régions). À droite, instance avec régions regroupées (30 aérodromes, 6 régions)

Instance	n	Valeur	Modèle 1 (MTZ) Borne	Temps (s)	noeuds	Modèle 2 (sous-tours) Borne	Temps (s)	noeuds
30_1 (fournie)	30	115	84.1	0.13	737	89.0	1.42	3 900
regroupée 30	30	141	94.1	1.50	13 537	95.0	123.96	1 481 190

À taille égale, les instances regroupées sont beaucoup plus difficiles. Dans une région, les aérodromes sont très proches, donc de petits cycles à l'intérieur d'une région coûtent très peu et permettent d'atteindre A_{min} et de valider la région sans être reliés au trajet. Il faut éliminer beaucoup de sous-tours, d'où plus de noeuds. Le rayon R de nos instances est aussi plus grand (plus de vols possibles), ce qui contribue à la difficulté. Les temps et bornes ici ne sont qu'indicatifs, les problèmes n'étant pas exactement les mêmes à résoudre.

5 Conclusion

Les deux modèles donnent les mêmes solutions. MTZ est le plus simple et le plus rapide dans notre implémentation, mais sa borne est plus faible. Le modèle 2 donne de meilleures bornes, mais la résolution répétée le pénalise sur les grandes instances. Enfin, quand les régions sont regroupées, les sous-tours sont nombreux et peu coûteux, et le problème devient beaucoup plus long à résoudre.

A Code Julia

Le code sauvegarde les chemins trouvés dans le fichier `parcours.txt` pour pouvoir les tracer ensuite avec le script `tracer_parcours.py`. Les résultats sont affichés dans le terminal et écrits dans `tableau.txt`.

Il faut installer JuMP et Gurobi pour exécuter le code, avec une licence « Named-User Academic » obtenue en créant un compte sur le site de Gurobi :

```
import Pkg
Pkg.add("JuMP")
Pkg.add("Gurobi_jll")
import Gurobi_jll
key = "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"   # la clé
run(`$(Gurobi_jll.grbgetkey()) $key`)
```

```
using JuMP
using Gurobi
include("lecture_distances.jl")
const GRB_ENV = Gurobi.Env()


# contraintes communes aux deux formulations
function modele(n, d, f, Amin, Nr, R, regions, D)
    model = Model(() -> Gurobi.Optimizer(GRB_ENV))
    set_silent(model)
    set_time_limit_sec(model, 600) # 10 minutes max

    @variable(model, x[1:n, 1:n], Bin) # x[i,j] = 1 si on vole de i a j

    # distance totale
    @objective(model, Min, sum(D[i,j]*x[i,j] for i in 1:n, j in 1:n))

    # pas de trajet de i vers i, ni de trajet plus long que R
    @constraint(model, [i = 1:n], x[i,i] == 0)
    @constraint(model, [i = 1:n, j = 1:n; D[i,j] > R], x[i,j] == 0)

    # on part de d et on arrive a f
    @constraint(model, sum(x[d,j] for j in 1:n) == 1)
    @constraint(model, sum(x[i,d] for i in 1:n) == 0)
    @constraint(model, sum(x[i,f] for i in 1:n) == 1)
    @constraint(model, sum(x[f,j] for j in 1:n) == 0)

    # autres aerodromes : autant d'entrees que de sorties, au plus une fois
    @constraint(model, [j = 1:n; j != d && j != f], sum(x[i,j] for i in 1:n) == sum(x[j,l] for
    ↪  l in 1:n))
    @constraint(model, [j = 1:n; j != d && j != f], sum(x[i,j] for i in 1:n) <= 1)

    # au moins Amin aerodromes, donc au moins Amin-1 trajets
    @constraint(model, sum(x) >= Amin - 1)

    # chaque region est visitee : au moins un vol arrive dans la region (sauf la region du
    ↪  depart)
    @constraint(model, [k = 1:Nr; !(d in regions[k])], sum(x[i,j] for i in 1:n, j in
    ↪  regions[k]) >= 1)

    return model, x
end


# formulation 1 : MTZ
function resoudre_mtz(fichier)
```

```
    n,d,f,Amin,Nr,R,regions,coords,D = readInstance(fichier)
    model, x = modele(n, d, f, Amin, Nr, R, regions, D)

    @variable(model, 1 <= u[1:n] <= n) # position de l'aerodrome dans le parcours
    @constraint(model, u[d] == 1)
    @constraint(model, [i = 1:n, j = 1:n; i != j], u[j] >= u[i] + 1 - n * (1 - x[i,j]))

    # borne par relaxation continue
    undo = relax_integrality(model)
    optimize!(model)
    borne = objective_value(model)
    undo()

    # branch-and-cut
    optimize!(model)
    valeur = has_values(model) ? objective_value(model) : NaN

    # on reconstruit le parcours en partant de d
    chemin = Int[]
    if has_values(model)
        chemin = [d]
        i = d
        while i != f
            i = findfirst(j -> value(x[i,j]) > 0.5, 1:n)
            push!(chemin, i)
        end
    end
    return n, borne, valeur, solve_time(model), node_count(model), termination_status(model),
    ↪  chemin
end


# formulation 2 : sous-tours ajoutes au fur et a mesure
# (il y a ~2^n contraintes de sous-tours, on ne peut pas toutes les ecrire : on resout sans,
↪  et on ajoute seulement celles qui sont violees)
function resoudre_sous_tours(fichier)
    n,d,f,Amin,Nr,R,regions,coords,D = readInstance(fichier)
    model, x = modele(n, d, f, Amin, Nr, R, regions, D)

    temps = 0.0
    noeuds = 0
    while true
        # resolution du probleme avec les sous-tours deja interdits
        set_time_limit_sec(model, max(1.0, 600 - temps))   # 10 minutes au total sur toute la
        ↪  boucle
        optimize!(model)
        temps += solve_time(model)
        noeuds += node_count(model)
        if termination_status(model) != MOI.OPTIMAL   # limite de temps atteinte
            return n, NaN, NaN, temps, noeuds, termination_status(model), Int[]
        end

        # on lit la solution avant de modifier le modele
        xv = value.(x)

        # on marque le chemin d -> f
        vu = falses(n)
        i = d
        vu[i] = true
        while i != f
            i = findfirst(j -> xv[i,j] > 0.5, 1:n)
            vu[i] = true
        end
```

```
104
105         # separation : un aerodrome visite hors du chemin est forcement dans un cycle
106         nb = 0      # nombre de sous-tours trouves a ce tour
107         for s in 1:n
108             if !vu[s] && sum(xv[s,j] for j in 1:n) > 0.5
109                 # on suit le cycle depuis s jusqu'a revenir en s : c'est l'ensemble S
110                 S = [s]
111                 vu[s] = true
112                 j = findfirst(l -> xv[s,l] > 0.5, 1:n)
113                 while j != s
114                     push!(S, j)
115                     vu[j] = true
116                     j = findfirst(l -> xv[j,l] > 0.5, 1:n)
117                 end
118                 # le cycle utilise |S| vols dans S : on en autorise au plus |S| - 1
119                 @constraint(model, sum(x[a,b] for a in S, b in S) <= length(S) - 1)
120                 nb += 1
121             end
122         end
123
124         # aucun sous-tour : la solution est un vrai trajet, donc optimale
125         if nb == 0
126             break
127         end
128     end
129     valeur = objective_value(model)
130     statut = termination_status(model)
131
132     # on reconstruit le parcours en partant de d (avant de relaxer le modele)
133     chemin = [d]
134     i = d
135     while i != f
136         i = findfirst(j -> value(x[i,j]) > 0.5, 1:n)
137         push!(chemin, i)
138     end
139
140     # borne par relaxation continue du modele final (avec les sous-tours ajoutes)
141     relax_integrality(model)
142     optimize!(model)
143     borne = objective_value(model)
144
145     return n, borne, valeur, temps, noeuds, statut, chemin
146 end
147
148
149 # toutes les instances
150 println("nom & n & formulation & borne & valeur & temps (s) & noeuds \\\\")
151 println("\\hline")
152 for fichier in readdir("instances"; join = true)
153     if endswith(fichier, "regions_50_2.txt")
154         nom = replace(basename(fichier), ".txt" => "")
155         for (formulation, resoudre) in [("MTZ", resoudre_mtz), ("Sous-tours",
            ↪  resoudre_sous_tours)]
156             n, borne, valeur, temps, noeuds, statut, chemin = resoudre(fichier)
157             ligne = string(nom, " & ", n, " & ", formulation, " & ", round(borne, digits=1), "
                ↪  & ", valeur, " & ", round(temps, digits=2), " & ", noeuds, " \\\\   % ",
                ↪  statut) # pour le tableau LaTeX
158             println(ligne)
159             println("% parcours : ", join(chemin, " -> "))
160             open("tableau.txt", "a") do io
161                 println(io, ligne)
162             end
163             open("parcours.txt", "a") do io
```

```
164                 println(io, nom, " ", formulation, " ", join(chemin, " "))
165             end
166         end
167     end
168 end
```